

LUCRAREA NR. 10

Compilatoare de C sub Linux

Scopul acestei lucrări este familiarizarea cu compilatorul de programe C sub Linux *gcc* precum și a modului de scriere și organizare a aplicațiilor în C sub Linux.

10.1. Compilatorul GCC

Compilatorul GCC sau GNU Compiler Collection este unul din cele mai utilizate și performante compilatoare pentru C/C++ (în special C, pentru C++ folosindu-se des compilatorul G++). Și aceasta aparține colecției de software GNU, adică este un program open source. GCC este un compilator extensibil, deși este dedicat pentru C dar cu extensii și pentru alte limbaje de programare cum ar fi: C++, Pascal, Fortran, Objective-C etc.

GCC este un program open source cu toate acestea executabilele generate de acesta nu trebuie să fie neapărat open source, chiar dacă include librăriile standard C sau C++.

Prima versiune GCC este datată în anul 1986. Pe parcursul anilor au apărut versiuni din ce în ce mai evolute, la început necesitând instalarea separată a compilatorului, mai apoi fiind incluse în distribuțiile Linux.

Sintaxa generală a compilatorului *gcc* este:

***gcc* opțiuni nume_fișier**

În linux, fișierle standard C au extensia “.c”. Pentru generarea unui fișier executabil dintr-un fișier standard C se va utiliza compilatorul *gcc* astfel:

gcc -o fișier_executabil fișier.c

Exemplul 1 - Să se compileze următorul program C (ex1.c):

```
#include <stdio.h>

int main()
{
    printf("Hello\n");
    return 0;
}
```

Rezolvare:

```
root@slax:~# gcc -o ex1 ex1.c
root@slax:~# ./ex1
Hello
```

```
root@slax:~#
```

În tabelul 10.1 sunt prezentate o parte din opțiunile care pot fi utilizate pentru compilarea fișierelor C.

Tabelul 10.1. Opțiunile compilatorului gcc

Opțiune	Descriere
-I/ <i>locatie</i>	caută fișierele “.h” în directorul <i>locatie</i> . Implicit, fișierele “.h” sunt căutate în directorul curent și în directoarele care conțin <i>header</i> ele pentru librăriile standard (/usr/include, /usr/src/include).
-L/ <i>locatie</i>	caută librăriile și în directorul <i>locatie</i> pe lângă directoarele standard “/lib”, “/usr/lib”.
-l <i>nume_librarie</i>	permite utilizarea funcțiilor din librăria <i>libnume_librarie</i>
-static	include în fișierul executabil toate funcțiile din librăriile utilizate în fișierul “.c” (<i>avantaj</i> : executabilul poate fi folosit pe alte sisteme fără a fi nevoie de librăriile cu funcții; <i>dezavantaj</i> : dimensiune considerabil mai mare a executabilului).
-c <i>fișier.c</i>	compilează un fișier C și generează un fișier obiect “ <i>fișier.o</i> ”.
-D <i>macro</i>	definește macro-ul cu numele <i>macro</i> (opțiune pentru preprocesor - identică cu “#define NUME valoare”)
Continuare în pagina următoare	

Tabela 10.1 – continuare

Opțiune	Descriere
-D <i>macro=valoare</i>	definește macro-ul cu numele <i>macro</i> și îi atribuie valoarea <i>valoare</i> .
-Wall	afișează toate mesajele de avertizare.

Anumite funcții utilizate în programele C necesită definirea unor fișiere *header* suplimentare iar în anumite cazuri compilarea fișierului necesită utilizarea unor librării suplimentare. De exemplu, pentru utilizarea funcțiilor matematice standard se va defini fișierul *header* "math.h" iar compilarea va necesita utilizarea opțiunii "-lm" care permite apelarea funcțiilor matematice din librăria "libm" asociată fișierului *header* "math.h".

Exemplul 2 - Să se scrie și să se compileze un program C (ex2.c) care calculează funcțiile *sinus* și *cosinus* ale unui unghi oarecare introdus de la tastatură:

```
#include <stdio.h>
#include <math.h>
void main(void)
{
    double unghi, sin_unghi, cos_unghi;
    printf("Unghi: ");
    scanf("%lf", &unghi);
    sin_unghi=sin((unghi*3.14)/180);
    cos_unghi=cos((unghi*3.14)/180);
    printf("sin(%f)= %f\n",unghi,sin_unghi);
    printf("cos(%f)= %f\n",unghi,cos_unghi);
}
```

Rezolvare:

```
root@slax:~# gcc -o ex2 ex2.c -lm
root@slax:~# ./ex2
Unghi: 30
sin(30.000000)= 0.499770
cos(30.000000)= 0.866158
root@slax:~#
```

10.2. Compilarea aplicațiilor cu surse multiple

În anumite cazuri, aplicațiile C complexe necesită o organizare mai riguroasă. De obicei, aplicațiile complexe conțin mai multe fișiere sursă ceea ce permite utilizatorului o mai bună înțelegere a modului de organizare a aplicației.

Orice aplicație *multi-sursă* conține un fișier sursă principal (de obicei *main.c*) și unul sau mai multe fișiere sursă secundare care conțin corpul funcțiilor definite de utilizator.

Pe lângă fișierele sursă, aplicațiile multi-sursă mai pot conține și fișiere de tip *header* (.h) care conțin definițiile funcțiilor utilizator.

Compilarea aplicațiilor de acest tip se face în doi pași:

- se compilează toate fișierele sursă utilizând opțiunea "-c" cu ajutorul căreia se generează câte un fișier obiect (cu extensia ".o") pentru fiecare fișier sursă.

```
gcc -c fisier1.c fisier2.c ... fisierN.c main.c
```

- se generează fișierul executabil utilizând opțiunea "-o" și fișierele obiect generate anterior.

```
gcc -o executabil fisier1.o ... fisierN.o main.o
```

Exemplul 3 - Să se scrie și să se compileze o aplicație multi-sursă care calculează suma și produsul a două numere întregi citite de la tastatură. Funcțiile *suma* și *produs* se vor scrie într-un fișier separat denumit *functii.c*.

main.c:

```
#include <stdio.h>
int main()
{
    int a,b;
    printf("a= ");
    scanf("%d",&a);
    printf("b= ");
    scanf("%d",&b);
```

```

        printf("Suma= %d\n", suma(a,b));
        printf("Produs= %d\n", prod(a,b));
        return 0;
    }

```

functii.c:

```

int suma(int i, int j)
{
    int s;
    s = i+j;
    return s;
}

int prod(int i, int j)
{
    int p;
    p = i*j;
    return p;
}

```

Rezolvare:

```

root@slax:~# gcc -c functii.c main.c
root@slax:~# gcc -o ex3 functii.o main.o
root@slax:~# ./ex3
a= 12
b= 13
Suma= 25
Produs= 156
root@slax:~#

```

10.3. Exerciții propuse

- 1) Să se scrie și să se compileze un program C care calculează media geometrică a N numere întregi introduse de la tastatură.

- 2) Să se scrie și să se compileze un program C care calculează funcțiile *tangentă* și *cotangentă* ale unui unghi oarecare.
- 3) Să se scrie și să se compileze un program C care calculează media aritmetică și media geometrică a N numere întregi introduse de la tastatură. Funcțiile de calcul ale celor două medii se vor scrie într-un fișier separat, denumit *medii.c*.

LUCRAREA NR. 11

Makefile-uri

Scopul acestei lucrări este înțelegerea modului de funcționare al utilitarului *make* și cunoașterea modului organizare a fișierelor *Makefile*.

11.1. Structura fișierelor *Makefile*

Aplicațiile complexe în C sub Linux conțin, în majoritatea cazurilor, un număr mare de fișiere sursă. De asemenea, în multe cazuri, compilarea fișierelor sursă necesită mai multe comenzi, fiecare având un număr mare de opțiuni. Dacă aplicația se află în stadiul de dezvoltare, orice testare va necesita compilarea tuturor surselor. Astfel, în anumite cazuri, procesul de compilare poate dura mai mult decât cel de programare.

Pentru a simplifica procesul de compilare, în special în cazul aplicațiilor cu surse multiple, se recomandă utilizarea fișierelor *Makefile* respectiv a utilitarului *make*.

Fișierele *Makefile* sunt fișiere text asociate utilitarului *make* care conțin macro-uri și comenzi pentru compilarea aplicațiilor (C, C++, etc).

Macro-urile dintr-un fișier *Makefile* reprezintă variabile pe care utilizatorul le poate folosi pentru compilarea aplicațiilor. Acestea li se pot atribui valori ca și în cazul scripturilor Shell, adică "variabilă=valoare". Macro-urile predefinite sunt:

- **CC** - numele compilatorului
- **OBJS** - numele fișierelor obiect (.o) care se vor genera
- **CFLAGS** - opțiunile utilizate pentru compilarea fișierelor sursă (ex: *-c -Wall*)
- **LFLAGS** - opțiunile utilizate pentru generarea executabilului (ex: *-lm -Wall*)

Pe lângă macro-uri, în fișierele *Makefile* se definesc și fișierele *target*, astfel:

```
target: dependinte
    <TAB> comanda
```

Makefile-uri

Target reprezintă numele fișierului care trebuie generat iar “dependinte” reprezintă numele fișierului sau fișierelor de care depinde fișierul *target*.

Fișierele Makefile se vor scrie după următoarele reguli: macro-urile se vor scrie întotdeauna cu litere mari; comenzile pentru generarea fișierelor target vor fi precedate de *TAB*; comenzile vor fi scrise pe baza macro-urilor, după următoarea sintaxă:

```
<TAB> $(MACROx) $(MACROy) optiuni $(MACROz) ... $(MACROxx)
```

exemplu:

```
$(CC) -o ex1 $(LFLAGS) $(OBJS)
```

Astfel, structura unui fișier Makefile este următoarea:

```
MACRO1=valoare
```

```
...
```

```
MACROn=valoare
```

```
target1: dependinte
```

```
<TAB> comenzi
```

```
...
```

```
targetN: dependinte
```

```
<TAB> comenzi
```

11.2. Compilarea aplicațiilor C cu utilitarul *make*

Compilarea aplicațiilor după scrierea fișierului Makefile se va face cu comanda *make target*.

Notă: *target* poate reprezenta și un cuvânt cheie folosit pentru operații suplimentare asupra aplicației. De exemplu, pentru ștergerea fișierelor obiect după generarea fișierului executabil se poate folosi:


```
clean:
    rm -rf *.o
```

Prin apelarea comenzii *“make clean”*, se vor sterge toate fişierele cu extensia *“.o”* din directorul curent.

Exemplul 1 - Să se creeze un fişier Makefile pentru compilarea următorului program C (ex1.c):

```
#include <stdio.h>
#include <math.h>
int main()
{
    int u=45;
    double res;
    res = sin((u*3.14)/180);
    printf("sin(%d) = %lf\n", u, res);
    return 0;
}
```

Makefile:

```
OBJS= ex1.o
CC= gcc
CFLAGS= -c -Wall
LFLAGS = -lm -Wall

ex1.o: ex1.c
    $(CC) $(CFLAGS) ex1.c
ex1: $(OBJS)
    $(CC) $(LFLAGS) $(OBJS) -o ex1
clean:
    rm -rf *.o ex1
```

compilare:

Makefile-uri

```
root@slax:~# make ex1
gcc -c -Wall ex1.c
gcc -lm -Wall ex1.o -o ex1
root@slax:~# ./ex1
sin(45) = 0.706825
root@slax:~#
```

Exemplul 2 - Să se creeze un fișier Makefile pentru compilarea următoarei aplicații cu surse multiple:

main.c:

```
#include <stdio.h>
double ma(int nr[20], int n);
double mg(int nr[20], int n);
int main() {
    double med_a, med_g;
    int i, n, nr[20];
    printf("n= ");
    scanf("%d", &n);
    for (i=0; i<n; i++) {
        printf("nr[%d]= ", i+1);
        scanf("%d", &nr[i]);
    }
    med_a = ma(nr, n);
    med_g = mg(nr, n);
    printf("Media aritmetica= %lf\n", med_a);
    printf("Media geometrica= %lf\n", med_g);
    return 0;
}
```

medii.c:

```
#include <math.h>
double ma(int nr[20], int n)
{
    int i;
```

```

        double res=0;
        for (i=0;i<n;i++)
        {
            res=res+nr[i];
        }
        return (res/n);
    }

double mg(int nr[20], int n)
{
    int i;
    double res=1;
    for (i=0;i<n;i++)
    {
        res=res*nr[i];
    }
    return sqrt(res);
}

```

Makefile:

```

OBJS= main.o medii.o
CC= gcc
CFLAGS= -c -Wall
LFLAGS = -lm -Wall

main.o: main.c
    $(CC) $(CFLAGS) main.c
medii.o: medii.c
    $(CC) $(CFLAGS) medii.c
ex2: $(OBJS)
    $(CC) $(LFLAGS) $(OBJS) -o ex2
clean:
    rm -rf *.o ex2

```

compilare:

```
root@slax:~# make ex2
gcc -c -Wall main.c
gcc -c -Wall medii.c
gcc -lm -Wall main.o medii.o -o ex2
root@slax:~# ./ex2
n= 3
nr[1]= 1
nr[2]= 2
nr[3]= 3
Media aritmetica= 2.000000
Media geometrica= 2.449490
root@slax:~#
```

11.3. Exerciții propuse

- 1) Să se scrie un program C care calculează rădăcinile reale sau complexe ale unei ecuații de gradul 2 de forma $ax^2 + bx + c = 0$, unde a , b și c sunt numere întregi introduse de la tastatură. Să se scrie un fișier *Makefile* pentru acest program.
- 2) Să se scrie un program C pentru calculul combinărilor de “n” luate câte “k” știind că: $C_n^k = \frac{n!}{(n-k)! * k!}$. Funcțiile *factorial* și *combinari* se vor scrie într-un fișier denumit *funcții.c* iar apelarea lor și citirea valorilor lui “n” și “k” se va face în fișierul *main.c*. Să se scrie un fișier *Makefile* pentru acest program.
- 3) Să se scrie un program C pentru calculul determinantului unei matrice 4x4 cu elemente numere reale introduse de la tastatură. Să se scrie un fișier *Makefile* pentru acest program.